

社区聚类完整算法流程

校园活动一站式服务平台
软件架构文档 · 算法视图专项材料

核心算法	K-Means 硬聚类 + PCA 二维降维
计算边界	Spring Boot 特征聚合 / FastAPI 聚类计算
输出结果	社区编号、成员归属、中心距离、二维坐标与聚类指标
版本日期	2026-07-16

本材料用于软件架构文档第 9 节“算法视图”，完整说明社区聚类模块从用户数据聚合、特征处理、K-Means 聚类、PCA 可视化到结果校验与版本化保存的全过程。

社区聚类完整算法流程

社区聚类模块的目标，是依据师生画像和真实活动参与行为，将用户自动划分为若干互斥社区，并为前端提供可解释、可追溯、可视化的社区结果。算法采用“Spring Boot 负责业务数据与持久化、FastAPI 负责数值计算”的职责分工。

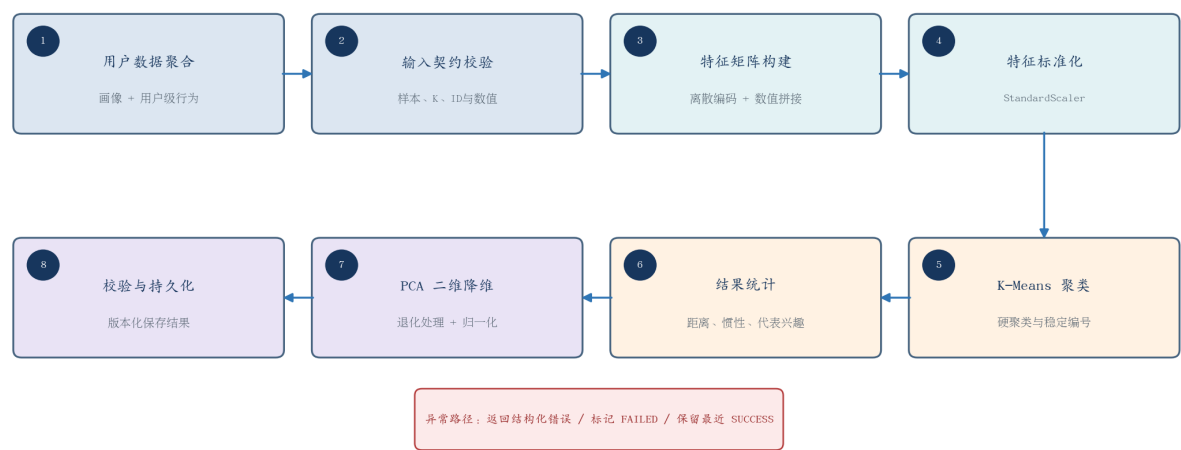


图 1 社区聚类完整算法流水线

总体原则

用户特征必须来源于现有业务数据；同一聚类版本中，一个用户只能属于一个社区；算法输出必须可复现、可校验、可版本化保存；失败运行不得覆盖最近一次成功结果。

流程目录

- 1. 用户数据聚合
- 2. 输入契约校验
- 3. 特征矩阵构建
- 4. 特征标准化
- 5. K-Means 硬聚类
- 6. 聚类结果统计
- 7. PCA 二维降维
- 8. 结果校验与版本化保存

1. 用户数据聚合

Spring Boot 以 `UserAccount` 为样本主表，批量读取用户画像和用户级活动行为，将其转换为确定性的 `FeatureSample`。聚合阶段只读数据库，不调用 Python，也不创建聚类运行记录。

特征类型	输入字段	业务来源
用户画像	兴趣、学院、年级、可参与时间	UserAccount
报名行为	报名总数、审核通过数	Signup
活跃行为	收藏数、签到数	Favorite / CheckInRecord
评价行为	评价数、平均评分	Feedback
类别偏好	各活动类别的通过报名次数	Signup + Activity.category

聚合阶段的关键处理：

- 排除管理员；教师、学生、组织者及其他非管理员用户均可纳入。
- 用户没有画像或行为记录时仍然保留，集合为空、计数为 0。
- 用户标识 `userId` 只做合法性校验，不进行 NFKC、大小写转换或替换。
- 兴趣、可参与时间和类别字段执行 NFKC、去首尾空白、去重和稳定升序。
- 平均评分仅使用 1 至 5 的有效评分；无有效评分时返回 `null`。
- 类别参与次数只由审核通过的报名产生，收藏、签到和评价不会重复增加类别次数。
- 所有计数先使用 `long` 聚合，转为 `int32` 前检查范围；溢出用户被排除并记录诊断。
- `FeatureSample` 最终按 `userId` 升序排列，保证相同数据库快照生成相同输入。

明确禁止

不得使用 `Activity.signupCount`、`Activity.favoriteCount` 等活动级展示计数作为用户行为特征；不得虚构浏览、点击、停留或搜索日志。

2. 输入契约校验

FastAPI 收到聚类请求后，首先验证样本结构和数值边界。校验失败时返回统一错误结构，不进入特征预处理和聚类计算。

- 1 样本列表必须非空。
- 2 聚类数量 K 满足 $2 \leq K \leq$ 有效用户数。
- 3 当前迭代 `randomState` 必填且固定为 42。
- 4 `userId` 必须唯一，不能在同一次请求中重复。
- 5 计数字段必须是 0 至 2,147,483,647 的严格整数。

- 6 浮点字段不得包含 NaN、Infinity 或 -Infinity。
- 7 不同特征行的数量必须不少于 K，避免无法形成 K 个有效簇。
- 8 特征字段、版本号和 schema 版本必须符合接口契约。

错误处理

非法样本、K 越界、特征版本不支持和计算失败均返回结构化 code、中文 message 与安全 details；错误响应本身必须符合标准 JSON。

3. 特征矩阵构建

每个用户的 FeatureSample 被转换为统一长度的数值向量。离散字段采用 One-Hot 或 Multi-Hot 编码，行为计数和评分直接作为数值列，最终构造 N×D 特征矩阵。

原始特征	编码方式	示例
兴趣标签	Multi-Hot	AI=1、篮球=0、音乐=1
可参与时间	Multi-Hot	周末上午=1、工作日晚间=0
学院	One-Hot	计算机学院=1，其余学院=0
年级	One-Hot	大三=1，其余年级=0
行为统计	数值列	报名数、签到数、评价数、平均评分
类别偏好	数值列	academic=2、club=0

矩阵定义：

特征矩阵

$X \in \mathbb{R}^{(N \times D)}$ ，其中 N 为有效用户数，D 为离散编码和数值特征拼接后的总维度。所有用户必须共享完全相同的列集合；某类别未参与时显式补 0。

用户	AI	篮球	计算机学院	大三	报名	签到	academic	club
U001	1	0	1	1	3	1	2	0
U002	0	1	0	1	2	2	0	2
T001	1	0	0	0	0	0	0	0

表 2 用户特征矩阵示意

4. 特征标准化

报名次数、评分、二值标签等字段的量纲和数值范围不同。若直接计算欧氏距离，数值较大的计数字段可能支配聚类结果，因此使用 StandardScaler 对每一列进行标准化。

$$z = (x - \mu) / \sigma$$

其中 x 表示第 i 个用户的第 j 个原始特征， μ 和 σ 分别是该特征列的均值和标准差。标准化后的矩阵用于 K-Means、聚类中心计算和 `distanceToCenter`。

标准化结果

标准化不会改变样本数和特征列数，只改变数值尺度，使不同类型的特征能够在同一距离空间内公平参与聚类。

5. K-Means 硬聚类

K-Means 将所有有效用户划分为 K 个互斥社区。它属于硬聚类：在同一聚类版本中，每个用户只能获得一个 `clusterNo`。

算法迭代过程：

- 1 使用请求中的 K 和 `randomState=42` 初始化 K 个聚类中心。
- 2 计算每个标准化用户向量到各聚类中心的欧氏距离。
- 3 将用户分配给距离最近的聚类中心。
- 4 按当前成员重新计算每个簇的均值中心。
- 5 重复“距离计算 - 成员分配 - 中心更新”，直到结果收敛或达到迭代上限。
- 6 对 K-Means 原始标签和中心进行规范化排序，生成稳定的 `clusterNo`。

$$\min \sum \|x - \mu(c)\|$$

目标函数表示：使所有用户到其所属社区中心的平方距离总和最小。固定 `randomState` 能保证同一行顺序下结果可复现；服务内部按 `userId` 排序和标签规范化进一步消除输入排列与标签交换带来的不稳定。

硬聚类约束

同一 run 中 $(runId, userId)$ 唯一；一个用户只能属于一个社区。同一用户可以在不同聚类版本中获得不同社区归属。

6. 聚类结果统计

K-Means 收敛后，聚类服务生成后续业务展示与质量评估所需的统计结果。

输出	含义
<code>clusterNo</code>	用户所属的稳定社区编号

输出	含义
distanceToCenter	用户在标准化特征空间中到所属中心的欧氏距离
inertia	所有用户到所属中心平方距离之和
memberCount	每个聚类的成员数量
topInterests	每个聚类中出现频率最高的代表兴趣
sampleCount / featureCount	本次聚类使用的样本数与特征维度

代表兴趣按照“频率降序、同频字符串升序”的确定性规则选取。Python只返回数值聚类结果和代表兴趣，社区名称、描述、颜色及业务ID由Spring Boot按固定规则生成。

7. PCA 二维降维

K-Means 在完整高维空间中执行，但前端散点图只能展示二维坐标。系统使用 PCA 提取主要变化方向，并将坐标分别归一化至 0-100。

有效秩	处理方式	坐标结果
rank = 0	所有样本在数值上完全相同	x=50, y=50
rank = 1	只计算一维 PCA	x 归一化, y=50
rank ≥ 2	计算二维 PCA	x、y分别归一化

单轴归一化：

$$\text{coord} = (\text{value} - \text{min}) / (\text{max} - \text{min}) \times 100$$

若坐标轴的范围为零，或处于预先规定的绝对/相对近零容差内，则该轴统一设为 50，避免浮点误差把极小差异放大为 0-100 的完整范围。所有输出必须为有限数，且满足 $0 \leq x,y \leq 100$ 。

可视化含义

二维坐标用于展示用户之间的相对接近程度，而不是地理坐标，也不直接参与社区归属判定；社区归属始终由完整标准化特征空间中的 K-Means 决定。

8. 结果校验与版本化保存

Spring Boot 接收 FastAPI 结果后，必须先进行跨请求、跨集合校验，再执行事务性保存。该阶段是算法结果进入业务系统前的最后一道边界。

保存前至少校验：

- runId、version、algorithm、K、randomState 和 featureSchemaVersion 与请求一致。

- 返回的用户集合与输入用户集合完全相同，每个用户恰好出现一次。
- 社区数量等于 K，clusterNo 集合完整且唯一。
- 每个成员引用的 clusterNo 确实存在。
- 各社区 memberCount 与实际成员数量一致。
- 坐标位于 0-100，中心距离和聚类指标为有限非负数。
- Python 不生成 communityId、社区名称、描述或颜色；这些由 Spring Boot 创建。

事务保存对象：

实体	主要内容
ClusteringRun	算法版本、K、状态、参数快照、指标、起止时间
Community	clusterNo、名称、描述、成员数、代表兴趣、颜色
CommunityMember	用户归属、二维坐标、中心距离、分配时间

成功路径

完整校验通过后，在同一事务中保存 Community 和 CommunityMember，并将当前运行标记为 SUCCESS。只有 SUCCESS 运行可以成为 latest。

失败路径

聚合失败、远程服务失败、响应校验失败或数据库保存失败时，将当前运行标记为 FAILED；不得留下部分社区数据，也不得覆盖最近一次成功结果。

算法流程总结

社区聚类算法形成了一条从业务数据到可视化社区结果的闭环：

阶段	职责主体	核心结果
用户数据聚合	Spring Boot	稳定、有序、可诊断的 FeatureSample 列表
契约校验与预处理	FastAPI	合法且统一维度的数值特征矩阵
标准化与K-Means	FastAPI	互斥社区归属和稳定 clusterNo
统计与PCA	FastAPI	中心距离、指标、代表兴趣和二维坐标
完整性校验	Spring Boot	确认响应与请求、社区及成员集合一致
版本化持久化	Spring Boot	可追溯的 Run、Community、Member 数据

核心架构价值

将业务数据聚合、数值算法和结果持久化解耦：Spring Boot维持权限、事务和业务一致性；FastAPI专注特征处理、K-Means和PCA。该设计便于独立测试算法、替换算法实现，并避免浏览器直接访问内部计算服务。

资料依据

- 《软件架构文档》模板第 9 节“算法视图”：要求对系统中的核心算法进行设计和说明。
- 校园活动一站式服务平台社区聚类模块当前架构与实现约束。
- K-Means、StandardScaler 与 PCA 的标准算法定义。